Phase 4 API Documentation

API Prefix: `/api/v1`

Phase 4 adds moderation, redaction, publication, shared-learning entitlements, product improvements, evidence, and audit retrieval.

All protected endpoints require an access token in the **`Authorization`** header. Roles are derived from active organisation memberships and are never trusted from the JWT.

Authentication and authorization

• Moderator: moderation queue, decisions, publication, manual unpublication, and audit retrieval.

• Administrator: entitlement management and moderator capabilities.

• Product owner: create improvements and change improvement status for permitted organisations.

• Client: retrieve entitled shared learning and improvements visible to the client organisation.

Moderation queue

`GET /moderation/observations`

• Purpose: list consent-granted observations without an existing moderation decision.

• Authentication: required.

• Role: moderator or administrator.

• Query: **`page`** (default 1), **`page_size`** (default 20, maximum 100).

• Success: **`200`**, pagination envelope containing queue observations.

• Errors: **`401`** unauthenticated, **`403`** insufficient role.

Moderation decision

`POST /moderation/observations/{observation_id}/decision`

```
{
  "decision": "approved",
  "reason": "Safe after redaction",
  "redacted_title": "Payment calculation issue",
  "redacted_content": "A payment calculation differed from the approved amount."
}
```

• Rejection may omit redacted fields; approval requires both non-empty redacted fields.

• Success: **`200`**.

• Errors: **`401`**, **`403`**, **`404 OBSERVATION_NOT_FOUND`**, **`409 MODERATION_DECISION_EXISTS`**, **`409 OBSERVATION_NOT_ELIGIBLE`**, and **`422 REDACTION_REQUIRED`**.

Publication

`POST /moderation/observations/{observation_id}/publish`

• Role: moderator or administrator.

• Requires active granted consent, an approved review, and redacted title/content.

• Success: **`200`** with entry id, observation id, status, and published time.

• Errors: **`401`**, **`403`**, **`404 OBSERVATION_NOT_FOUND`**, **`409 PUBLICATION_NOT_ELIGIBLE`**, and **`409 PUBLICATION_ALREADY_EXISTS`**.

`DELETE /shared-learning/{entry_id}`

• Role: moderator or administrator.

• Optional query: **`reason`** (maximum 500 characters).

• Sets the entry to unpublished and records an audit event.

• Success: **`200`**.

• Errors: **`401`**, **`403`**, **`404 SHARED_LEARNING_NOT_FOUND`**, **`409 SHARED_LEARNING_ALREADY_UNPUBLISHED`**.

Product entitlement

`POST /entitlements/product`

```
{
  "organisation_id": "uuid",
  "product_id": "uuid",
  "is_active": true
}
```

• Role: administrator only.

• Creates or updates an organisation/product entitlement.

• Success: **`200`**.

• Errors: **`401`**, **`403`**, **`404 PRODUCT_NOT_FOUND`**, **`404 ORGANISATION_NOT_FOUND`**.

Shared learning retrieval

`GET /shared-learning`

• Role: authenticated active client.

• Query: optional **`product_id`**, **`page`**, and **`page_size`**.

• Returns only published entries. Product-associated entries require an active organisation product entitlement; product-free entries are visible to active client organisations.

• Success: **`200`** with pagination.

• Errors: **`401`**, **`403`**.

`GET /shared-learning/{entry_id}`

• Returns the entry only when published, entitled, and its observation still has granted consent.

• Success: **`200`**.

• Errors: **`401`**, **`403`**, **`404 SHARED_LEARNING_NOT_FOUND`**.

Consent withdrawal immediately marks every published shared entry for the observation as unpublished and writes withdrawal/unpublication audit events in the same database transaction.

Product improvements and evidence

POST /product-improvements

```
{
  "title": "Improve calculation",
  "description": "Adjust calculation behavior based on evidence.",
  "product_id": "uuid",
  "organisation_id": "uuid",
  "evidence_entry_ids": ["uuid"]
}
```

• Role: administrator or product owner.

• Evidence entries must be published, belong to the improvement product, and have active consent.

• Success: **201**.

• Errors: **401**, **403**, **404 PRODUCT_NOT_FOUND**, and **409 INELIGIBLE_EVIDENCE**.

GET /product-improvements

• Query: optional **product_id**, **page**, **page_size**.

• Administrators, moderators, product owners, and employees receive general visibility according to existing tenant rules. Clients receive improvements with no organisation scope or improvements scoped to their organisation.

• Evidence output is marked active only while publication and consent remain active; withdrawn evidence has **active: false**.

PATCH /product-improvements/{improvement_id}/status

```
{
  "status": "under_review",
  "reason": "Starting review"
}
```

• Role: administrator or product owner.

• Valid transitions: proposed to under_review/rejected; under_review to planned/rejected; planned to in_progress/rejected; in_progress to completed/rejected.

• Terminal proposed/in-review/planned/in-progress states follow the documented transitions; completed and rejected cannot transition further.

• Success: **200**.

• Errors: **401**, **403**, **404 IMPROVEMENT_NOT_FOUND**, **409 INVALID_IMPROVEMENT_TRANSITION**.

Audit events

GET /audit-events

• Role: moderator or administrator.

• Query: **page**, **page_size**.

• Success: **200** with audit metadata, excluding passwords, tokens, and original confidential observation text.

• Errors: **401**, **403**.

• Audit records are append-only; there is no public update or delete endpoint.

Database migration

Migration: **`phase4_platform_workflows`**

Creates moderation reviews, shared-learning entries, organisation/product entitlements, improvements, improvement evidence, and audit events. MySQL migration verification completed with Alembic head **`phase4_platform_workflows`**.

Tests and limitations

• Added **`tests/test_phase4.py`** covering approval/publication, unauthorized moderation and entitlement management, withdrawal unpublication, evidence, improvement status changes, and non-entitled shared-learning access.

• Full suite: 49 passing tests.

• Tests use in-memory SQLite. MySQL migration was verified separately with Alembic against the configured database.